

На правах рукописи



Сиднев Алексей Александрович

**Технология быстрого прототипирования программ на основе
многовариантных программных компонент**

05.13.11 – Математическое и программное обеспечение вычислительных
машин, комплексов и компьютерных сетей

АВТОРЕФЕРАТ

диссертации на соискание ученой степени
кандидата технических наук

Санкт-Петербург – 2016

Работа выполнена на кафедре математического обеспечения ЭВМ факультета вычислительной математики и кибернетики Нижегородского государственного университета им. Н.И. Лобачевского.

Научный руководитель: **Гергель Виктор Павлович**
профессор, доктор технических наук

Официальные оппоненты: **Аветисян Арутюн Ишханович**
доцент, доктор физико-математических наук,
директор Института системного
программирования Российской академии наук

Терехов Андрей Николаевич
профессор, доктор физико-математических наук,
заведующий кафедрой системного
программирования математико-механического
факультета ФГБОУ ВО «Санкт-Петербургский
государственный университет»

Ведущая организация: **ФГБОУ ВО «Московский государственный
университет имени М.В.Ломоносова»**

Защита состоится «26» мая 2016 г. в 15:30 на заседании диссертационного совета Д 212.227.06 при Санкт-Петербургском национальном исследовательском университете информационных технологий, механики и оптики по адресу: 197101, Санкт-Петербург, Кронверкский пр., д.49., ауд. 431.

С диссертацией можно ознакомиться в библиотеке Санкт-Петербургского национального исследовательского университета информационных технологий, механики и оптики по адресу: 197101, Санкт-Петербург, Кронверкский пр., д.49 и на сайте http://fppo.ifmo.ru/?page1=16&page2=52&page_d=1&page_d2=159125.

Автореферат разослан «31» марта 2016 г.

Ученый секретарь
диссертационного совета Д 212.227.06
к.ф.-м.н., доцент

Холодова С.Е.

ОБЩАЯ ХАРАКТЕРИСТИКА РАБОТЫ

Актуальность темы диссертационной работы. Разработка программной системы – это сложная техническая и аналитическая задача. Методы разработки программного обеспечения являются областью активных научных исследований. Можно выделить работы советских, российских и зарубежных ученых М.И. Кахро, А.П. Калья, Э.Х. Тыугу, В.М. Глушков, В.Э. Малышкин, Б. Мейер, Дж.Р. Корбин, Дж. Сигел и др. Подход к разработке программ, использующий модули и компоненты, которые оформляются в библиотеки, является на данный момент общепризнанным.

Одна из основных проблем модульной разработки – отсутствие стандартов на интерфейсы модулей. Разработчик библиотеки сам определяет удобные ему структуры хранения данных и интерфейсы функций, которые их обрабатывают. В результате, каждая реализация получается уникальной и возникает сложность, связанная с заменой используемых библиотек.

Разработка программной системы включает в себя обоснованный выбор используемых технологий и библиотек среди множества обладающих близкой функциональностью. Существует широкий класс базовых задач, таких как решение СЛАУ, вычисление преобразования Фурье и многих других, которые используются во многих прикладных проектах. Развитие вычислительных систем, программных платформ, технологий разработки программ и многообразие методов решения этих задач привело к тому, что для них разработано множество библиотек. Принять окончательное решение об используемой библиотеке можно только после детального анализа и разработки прототипа системы с использованием множества реализаций. Разработка подобного прототипа в части использования библиотек требует решения следующих задач: выбор наиболее оптимальной библиотеки под текущие задачи проекта; поддержка нескольких библиотек; замена и/или добавление новых библиотек.

Каждая библиотека имеет свою сложность внедрения, эффективность реализации, удобство использования структур данных и поддержку различных программно-аппаратных платформ. Задача выбора библиотеки часто является многокритериальной, в связи с чем приходится идти на компромисс, выбирая некоторое «среднее» решение. Окончательный выбор одной или нескольких реализаций может быть выполнен только после разработки прототипа и проведения экспериментов по решению наиболее типичных задач проекта. При эволюционной разработке программной системы могут возникнуть задачи, которые нельзя решить, используя текущие библиотеки, например, переход на новую программно-аппаратную платформу, повышение эффективности, увеличение функциональных возможностей и т.д., тогда может возникнуть необходимость замены и/или добавления новых библиотек.

Указанные выше моменты и определяют актуальность диссертационной работы.

Предметом исследования являются методы прототипирования программного обеспечения, библиотеки решения вычислительных задач, методы и алгоритмы прогноза времени выполнения программ.

Основной **целью работы** является разработка технологии быстрого прототипирования для сокращения времени разработки программного обеспечения при условии необходимости выбора из множества близких по функциональности компонент (вычислительных библиотек). Предлагаемый подход должен обеспечивать бинарную совместимость библиотек и единообразие программного кода. Для описания алгоритмов и использующихся структур данных необходимо предложить макроязык, который бы позволил устранить зависимость описания от конкретного языка программирования и исключить модификацию существующего кода. Среди всех реализаций библиотек должна использоваться наиболее эффективная для текущей программно-аппаратной платформы, выбираемая автоматически, или любая, выбранная пользователем. Для задачи автоматического выбора необходимо привести формальную постановку задачи и предложить метод её решения.

Основные **задачи** диссертационной работы:

1. Разработать метод построения оценки времени выполнения программ.
2. Предложить технологию быстрой разработки прототипов при условии необходимости выбора из множества близких по функциональности компонент.
3. Разработать и реализовать программный комплекс, обеспечивающий сборку и исполнение программ, реализованных с использованием предлагаемого подхода быстрой разработки прототипов.
4. Подтвердить практическую полезность предложенного подхода результатами исследования трудозатрат при разработке и модификации программного обеспечения.
5. Внедрить результаты работы в программную систему «Кристалл», предназначенную для решения задач планирования процесса изготовления больших интегральных схем с субмикронными топологическими нормами, в НИИ измерительных систем им. Ю.Е. Седакова.

Научная новизна. При выполнении работы получены следующие основные новые результаты:

- Разработан новый метод оценки времени выполнения программ, основанный на методах машинного обучения.
- Предложена новая технология быстрой разработки прототипов, позволяющая сократить время разработки при условии необходимости выбора из множества близких по функциональности компонент.

Практическая ценность заключается в следующем.

Разработан и реализован программный комплекс, обеспечивающий сборку и исполнение программ, реализованных с использованием предлагаемого подхода. Программный комплекс может использоваться для разработки прототипов программ, которые требуют поддержки нескольких вычислительных библиотек.

лиотек и автоматического (или ручного) выбора наиболее эффективной реализации для заданных параметров запуска.

Практическая полезность предложенного подхода подтверждена результатами исследования трудозатрат при разработке и модификации программного обеспечения.

Методология и методы исследования. Методологическую основу диссертации составили принципы формализации, обобщения, проведение экспериментальных исследований и анализ их результатов. Работа основана на модульном подходе разработки программного обеспечения, подходах, применяющихся при решении задачи миграции программного обеспечения, а так же методах машинного обучения, обеспечивающих построение оценок времени выполнения программ.

На защиту выносятся следующие положения:

1. Метод построения оценки времени выполнения программ, основанный на методах машинного обучения.
2. Технология быстрой разработки прототипов на основе многовариантных программных компонент.
3. Программный комплекс сборки и исполнения прототипов программ.
4. Результаты численных экспериментов по оценке качества предсказания времени выполнения алгоритмов.

Обоснованность и достоверность научных положений и выводов, полученных в диссертации, подтверждается строгостью постановки задачи выбора наиболее эффективной реализации алгоритма, результатами экспериментов и их статистическим анализом, обоснованностью применения математического аппарата, обсуждением на научных конференциях и экспертизой при публикации в научной печати.

Внедрение результатов работы. Исследования по теме диссертации выполнялись при поддержке Министерства образования и науки РФ (тема Н-465-99) в рамках НИЛ "Суперкомпьютерные технологии в решении наукоемких прикладных задач". Работа поддержана грантом (соглашение от 27 августа 2013г. № 02.В.49.21.0003 между МОН РФ и ННГУ).

Практическая полезность разработанного комплекса подтверждена апробацией при создании программной системы «Кристалл», предназначенной для решения задач планирования процесса изготовления больших интегральных схем с субмикронными топологическими нормами, в НИИ измерительных систем им. Ю.Е. Седакова.

Апробация работы. Результаты диссертационной работы докладывались и обсуждались на следующих конференциях и семинарах: конференция «SYRCoSE» (Нижний Новгород, 2010), международная конференция «Высокопроизводительные параллельные вычисления на кластерных системах» (Пермь, 2010, 2014), международная суперкомпьютерная конференция «Научный сервис в сети Интернет» (Новороссийск, 2014), конференция «Разработка ПО» CEE-SECR (Москва, 2014), семинар Института системного программирования

РАН (Москва, 2014), семинар научно-исследовательского вычислительного центра МГУ (Москва, 2015), научный семинар «Методы суперкомпьютерного моделирования» ИКИ РАН (Таруса, 2015), научные конференции и семинары Нижегородского государственного университета (Нижний Новгород, Саров).

Публикации. По теме диссертации опубликовано 12 работ. Среди них 11 печатных работ, включая две статьи в изданиях, рекомендованных ВАК. Имеется одно свидетельство о государственной регистрации программ для ЭВМ. Список публикаций приведен в конце автореферата.

Свидетельства о регистрации программ для ЭВМ. В рамках диссертационной работы получено свидетельство о регистрации программы для ЭВМ.

Личный вклад соискателя. Постановка задачи и методика исследования принадлежат руководителю. Соискателю принадлежит разработка новой формулировки задачи построения оценки времени выполнения программ, метода её решения, разработка программного обеспечения, выполнение исследования трудозатрат, обработка и интерпретация результатов.

Структура и объем работы. Диссертационная работа состоит из введения, 4 глав, заключения, списка использованной литературы из 87 наименований и приложений. Основная часть работы изложена на 122 страницах текста, содержит 38 рисунков и 13 таблиц.

КРАТКОЕ СОДЕРЖАНИЕ РАБОТЫ

Во **введении** отражена актуальность темы диссертационного исследования, определяются цели и задачи исследования, показана научная новизна и практическая ценность диссертационной работы.

В **первой главе** представлена формулировка задачи быстрой разработки прототипов при условии выбора из множества близких по функциональности компонент. Выделены основные проблемы, возникающие при разработке прототипов данного типа. Выполнен аналитический обзор актуальных методов решения проблемы миграции, выбора и поддержки нескольких библиотек при разработке программного обеспечения.

В разделе 1.2 представлен обзор библиотек разработки прикладных программ на примере задачи решения СЛАУ.

В работе выполнен обзор методов решения и форматов хранения СЛАУ. Рассмотрены 20 математических библиотек, предназначенных для решения СЛАУ, с перечислением методов, которые они реализуют (прямые и/или итерационные). Также рассмотрены форматы хранения матриц (плотные и/или разреженные), поддерживаемые этими библиотеками, даты их последнего обновления, доступность исходных кодов, возможность параллельных вычислений и программные платформы, под которые реализованы библиотеки.

Выполнен анализ взаимозаменяемости программных пакетов на примере базовых операций линейной алгебры (BLAS) и вычисления преобразования Фурье. Анализ показал, что даже для этих широко применяемых задач наблю-

даются различия в реализациях интерфейсов. Для библиотек решения СЛАУ, где разнообразие методов и структур данных велико, эта проблема ещё более актуальна.

В разделе 1.3 представлен обзор существующих подходов к решению задачи миграции, выбора и поддержки нескольких библиотек: стандартизация, шаблоны проектирования, средства автоматизации сборки, средства автоматической трансформации программ. Применение рассмотренных подходов требует специальных навыков, а по части рассмотренных направлений ведутся дальнейшие исследования. В работе рассматривается решение, объединяющее в себе часть рассмотренных подходов, что позволяет устранить их недостатки и снизить остроту проблем, связанных с разработкой прототипов.

Во **второй главе** рассматривается технология быстрой разработки прототипов программ при необходимости выбора из множества близких по функциональности компонент.

В разделе 2.1 вводится терминология, используемая в работе.



Рисунок 1. Основные положения технологии быстрого прототипирования

В разделе 2.2 приводится общая характеристика предлагаемого подхода к разработке прототипов программ.

В качестве компонент рассматриваются вычислительные библиотеки, решающие задачи одного класса. Суть предлагаемой технологии прототипирования может быть сформулирована в виде следующих основных положений (рис. 1):

1. Для библиотек, предназначенных для решения выбранного класса задач, разрабатываются стандартные интерфейсы.

2. Реализации библиотек должны предоставлять соответствующие принятым стандартам интерфейсы или адаптеры для них.

3. Программы, разрабатываемые с использованием некоторой библиотеки, должны обращаться к ней через стандартные интерфейсы.

4. Среди всех реализаций библиотек должна использоваться наиболее эффективная для текущей программно-аппаратной платформы или любая выбранная пользователем.

В работе рассмотрены проекты стандартов для двух областей: базовые операции линейной алгебры, глобальная оптимизация.

В разделе 2.3 приводится общая схема предлагаемого подхода, которая включает этапы разработки и исполнения программ, разработанных с использованием технологии быстрого прототипирования (рис. 2).



Рисунок 2. Схема сборки и исполнения программ, разработанных с использованием технологии быстрого прототипирования

Разработка приложения, которое поддерживает множество реализаций библиотек — сложная задача. Разработка начальной версии, которая использует только одну из библиотек, является одним из вариантов разработки программы. Далее, с помощью специального макроязыка в программе выполняется описание всех вызовов функций из библиотеки.

Макроязык является декларативным специализированным языком, который предназначен для описания структур данных, использующихся в вызове функций из библиотек. Описание каждого независимого вычислительного участка программы реализуется отдельно, формируя макрокоманду. В работе определён синтаксис макроязыка в формате расширенной формы Бэкуса—Наура (РФБН). Для сборки приложений, содержащих макрокоманды, необходимо выполнить трансляцию исходных кодов с помощью макропроцессора. Полученный исполняемый модуль содержит макрорасширения, соответствующие каждой макрокоманде. Исполнение макрорасширения состоит из следующих этапов:

1. Определение функции из библиотеки, которая будет использоваться для выполнения алгоритма, соответствующего макрокоманде (выполняется планировщиком).

2. Преобразование входных данных к формату, используемому в библиотеке (выполняется конвертером).

3. Вызов реализации алгоритма из выбранной библиотеки (выполняется адаптером).

4. Преобразование выходных данных к формату, используемому в программе пользователя (выполняется конвертером).

В разделе 2.4 подробно рассматриваются все элементы технологии быстрой разработки прототипов программ и метод выбора наиболее эффективной реализации алгоритма.

Задача выбора наиболее эффективной реализации алгоритма решается планировщиком, который должен выполнять выбор как во время исполнения программы, так и на этапе её сборки. В качестве критерия эффективности реализации алгоритма будем рассматривать время её выполнения. Тогда задача планировщика состоит в оценке времени выполнения всех доступных реализаций и выбора самой быстрой из них.

Пусть имеется r реализаций алгоритмов, которые предназначены для решения одной задачи. Будем предполагать, что все реализации имеют одинаковый набор параметров. Например, для задачи умножения матриц в качестве параметров будут выступать размерности матриц. Обозначим через $P = (p_1, p_2, \dots, p_m)$ – вектор входных параметров алгоритма, через $X = (x_1, x_2, \dots, x_k)$ – вектор характеристик вычислительной системы, тогда время выполнения i -ой реализации алгоритма y_i при заданных параметрах P на вычислительной системе с характеристиками X задается в виде $y_i = f_i(P, X)$, где f_i – неизвестная функциональная зависимость, $i = \overline{1, r}$. Выбор наиболее эффективной реализации алгоритма i^* заключается в нахождении реализации, решающей задачу за минимальное время:

$$i^* = \operatorname{argmin}_{i=1..r} f_i(P, X). \quad (1)$$

Для решения задачи выбора (1) необходимо решить задачу оценки времени выполнения, состоящую в построении функции $g_i(P, X)$, которая наилучшим образом аппроксимирует f_i . Классические подходы выполняют построение оценки времени выполнения программ на основании истории запусков этих программ на одной вычислительной системе, т.е. функция f_i зависит только от параметров алгоритма P . При таком подходе необходимо проводить тестирование реализаций на каждой целевой вычислительной системе, что может потребовать существенного времени. В работе формулируется другая постановка задачи, позволяющая выполнять оценки без проведения экспериментов на целевой вычислительной системе.

Критерием качества построенной оценки выберем следующую функцию потерь:

$$Q(g_i) = \sum_{(P, X) \in T} (g_i(P, X) - f_i(P, X))^2, \quad (2)$$

где T – тестовая выборка, на которой известны значения функции $f_i(P, X)$. Таким образом, задача оценки времени выполнения реализации алгоритма на конкретной вычислительной системе заключается в построении функции g_i^* , минимизирующей функцию потерь:

$$g_i^* = \operatorname{argmin}_{g_i} \sum_{(P, X) \in T} (g_i(P, X) - f_i(P, X))^2. \quad (3)$$

Для построения функции g_i^* предлагается использовать информацию о времени выполнения реализации алгоритма на n вычислительных системах. Совокупность

$$T = (X_i, (y_{i,j}, P_{i,j})_{j=1}^{e_i})_{i=1}^n \quad (4)$$

представляет собой обучающую выборку, где e_i – количество экспериментов, которое было проведено на i -ой вычислительной системе.

Задача оценки времени выполнения (3) является частным случаем задачи восстановления регрессии, которая на практике решается широким кругом методов машинного обучения. Для того чтобы воспользоваться каким-либо методом машинного обучения, необходимо определить параметрическое семейство отображений $g(P, X, \theta)$, где $\theta \in \Theta$, $g: P \times X \times \Theta \rightarrow \mathbb{R}^+$ – фиксированная функция. Задача (3) обладает рядом особенностей, которые затрудняют построение параметрического семейства и, как следствие, не позволяют использовать существующие методы машинного обучения:

1. Зависимость времени выполнения реализации алгоритма от параметров вычислительной системы сложно представить в виде аналитической функции с достаточной точностью.

2. Оценку времени выполнения необходимо выполнять не только в интервале изменения значений параметра P обучающей выборки, но и за его пределами (например, для параметров, соответствующих задаче большей размерности), т.е. требуется выполнять экстраполяцию.

В работе предлагается использовать алгоритм, который является комбинацией методов восстановления линейной и нелинейной регрессии.

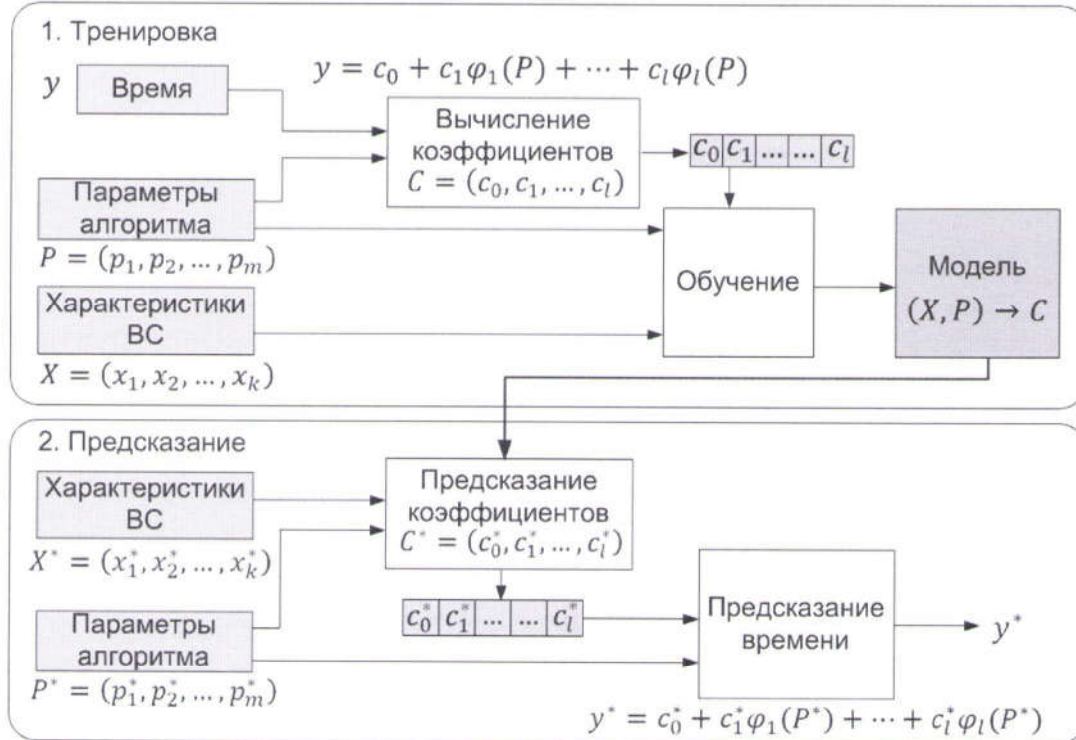


Рисунок 3. Алгоритм тренировки и предсказания

Для оценки времени выполнения программ на практике широко используются асимптотические оценки сложности алгоритмов. Будем предполагать, что для каждой реализации алгоритма известна его асимптотическая оценка сложности, которую можно представить в виде

$$h(P, C) = c_0 + \sum_{i=1}^l c_i \varphi_i(P), \quad (5)$$

где φ_i – произвольная функция от параметров алгоритма. Тогда время выполнения реализации алгоритма y_i может быть представлено в виде функции $h(P, C)$ с заданными коэффициентами $C = (c_0, c_1, \dots, c_l)$. Отметим, что коэффициенты C зависят от значений параметров P , т.к. реализации могут иметь разную степень оптимизации в зависимости от размерности задачи.

Алгоритм тренировки состоит из двух этапов (рис. 3):

1. На каждой вычислительной системе X для каждого значения параметров P из обучающей выборки определяются коэффициенты C функции $h(P, C)$ с заданной величиной согласованности W . Алгоритм восстановления коэффициентов C применяется для каждой вычислительной системы $i = \overline{1, n}$ отдельно:

Шаг 1. Задаём тренировочную выборку $T_i = (y_{i,j}, P_{i,j})_{j=1}^{e_i}, s = 1$.

Шаг 2.1. Если $s > e_i$, то все точки рассмотрены, переходим к шагу 3, иначе с помощью линейного метода наименьших квадратов (МНК) восстанавливаем коэффициенты $C = (c_0, c_1, \dots, c_l)$ функции $h(P, C)$ на выборке T_i .

Шаг 2.2. Оцениваем согласованность построенной линейной модели и экспериментальных данных T_i , используя коэффициент детерминации с поправкой на число параметров модели

$$\overline{R^2} = R^2 - (1 - R^2) \frac{l}{e_i - l - 1}, \quad (6)$$

где R^2 – коэффициент детерминации.

Шаг 2.3. Если значение $\overline{R^2}$ больше заданного W , то $PC_s = C$, рассматриваем следующую точку ($s = s + 1$) и переходим к шагу 2.1. Иначе переходим к шагу 2.4.

Шаг 2.4. Если $h(P_{i,s}, C) > y_{i,s}$, то из T_i исключаем $\arg \max_{j=1..e_i} (h(P_{i,j}, C) - y_{i,j})$, иначе из T исключаем $\arg \min_{j=1..e_i} (h(P_{i,j}, C) - y_{i,j})$. $e_i = e_i - 1$, переходим на шаг 2.1.

Шаг 3. Кластеризация найденных коэффициентов $PC_{j=1}^{e_i}$ с использованием DBSCAN.

2. Строится обучающая выборка из коэффициентов, полученных на предыдущем шаге, и соответствующих характеристик вычислительной системы и параметров алгоритма $(X, P) \rightarrow C$. Выполняется обучение нелинейной модели на полученной выборке.

Алгоритм предсказания также состоит из двух этапов (рис. 3):

1. По характеристикам вычислительной системы X^* и параметрам алгоритма P^* выполняется восстановление с использованием натренированной модели коэффициентов $C^* = (c_0^*, c_1^*, \dots, c_l^*)$.

2. По параметрам алгоритма P^* и полученным на предыдущем шаге коэффициентам C^* вычисляется время выполнения алгоритма

$$y^* = c_0^* + c_1^* \varphi_1(P^*) + \dots + c_l^* \varphi_l(P^*). \quad (7)$$

Таким образом, предлагаемое решение задачи (3) состоит в том, что выполняется предсказание не значения функции времени в точке (X, P) , а коэффициентов функции асимптотической сложности, которая проходит через заданную точку. В результате, предлагаемое решение способно выполнять экстраполяцию за пределами тренировочных данных, а использование нелинейной модели обучения позволяет восстанавливать сложные зависимости времени выполнения программ от характеристик вычислительной системы.

В диссертации приведены результаты экспериментов по предсказанию времени выполнения для четырёх алгоритмов: умножение вещественных плотных матриц, сортировка, быстрое преобразование Фурье, решение СЛАУ методом исключения Гаусса. Предварительно была сформирована экспериментальная база, состоящая из времени выполнения реализаций алгоритмов на **84 вычислительных системах** архитектуры x86 с общей памятью (мобильные, настольные, серверные) с использованием библиотек Intel MKL, OpenBLAS, Intel TBB, FFTW. В качестве характеристик вычислительных систем (ВС) использовалось **133 признака**: пиковая теоретическая производительность, количество ядер, процессоров, тактовая частота системной шины, процессора в обычном режиме и turbo-режиме, наличие Hyper-Threading, тип ядра процессора, производитель процессора, информация по каждому уровню кэш-памяти процессора (размер, количество портов доступа, общий для всех ядер или отдельный), информация по подсистеме памяти (количество каналов доступа, тайминг), поддерживаемый набор инструкций, величина пропускной способности блока памяти (от 8 КБ до 64 МБ) с различными вариантами доступа. 133 признака для 84 вычислительных систем являются избыточными. Для задачи умножения матриц было выявлено, что наиболее значимыми характеристиками вычислительной системы являются: пиковая теоретическая производительность, последовательная запись блока памяти размером 2 МБ, случайный доступ на запись в блок памяти размером 2 МБ и случайный доступ на чтение в блок памяти размером 2 МБ.

В диссертации представлено сравнение широко используемых методов оценки времени выполнения с предлагаемым подходом. На рис. 4 представлены результаты оценки времени выполнения в виде диаграмм размаха для каждой ВС с использованием предлагаемого подхода на задаче матричного умножения из библиотеки Intel MKL при обучении на матрицах малой размерности до 1000×1000 (36 экспериментов на каждой ВС) и предсказании времени для матриц размерности до 5000×5000 (200 экспериментов на каждой ВС). Каждая вычислительная система последовательно выступала в качестве тестовой выборки, а в качестве обучающей – все остальные. Результаты представлены

как для случая, когда в обучающей выборке отсутствуют данные с целевой ВС, так и для случае когда в обучающую выборку добавлено 5 результатов экспериментов, проведенных на целевой ВС.

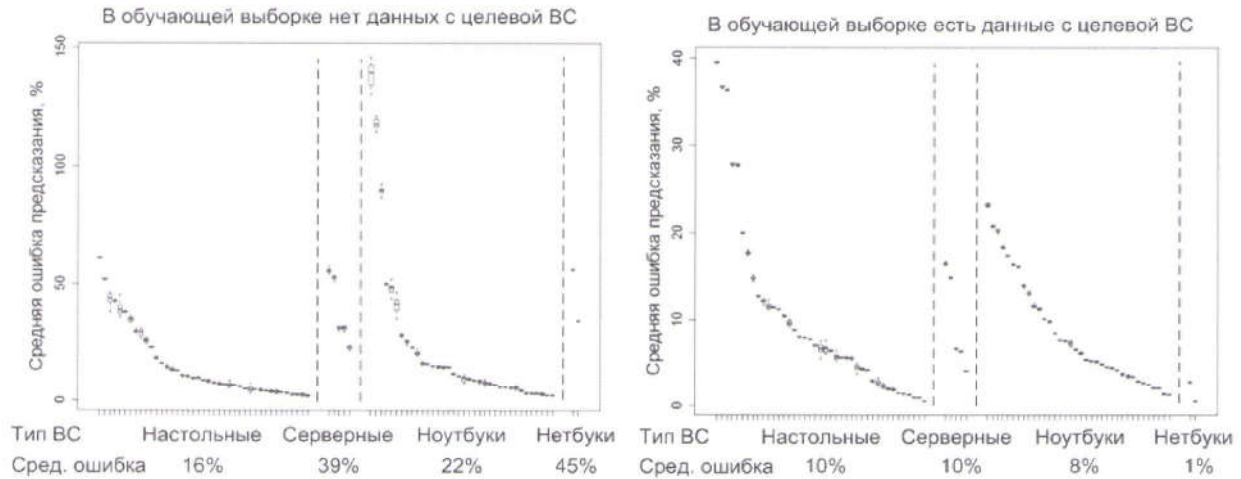


Рисунок 4. Результаты предсказания времени умножения матриц с использованием предлагаемого подхода

На рис. 4 видно, что оценка, полученная с использованием предлагаемого подхода, обладает ошибкой около 19% для систем, широко представленных в обучающей выборке (настольные и ноутбуки). Наличие экспериментальных данных с целевой вычислительной системы в обучающей выборке, позволяет сократить среднюю ошибку до 10% для этих систем.

В работе приведены результаты использования ряда широко используемых методов машинного обучения для решения задачи (3), таких как случайный лес, метод опорных векторов с нелинейным ядром, нейронные сети. Средняя относительная ошибка предсказания времени выполнения составила более 95%, что делает их непригодными для решения подобных задач.

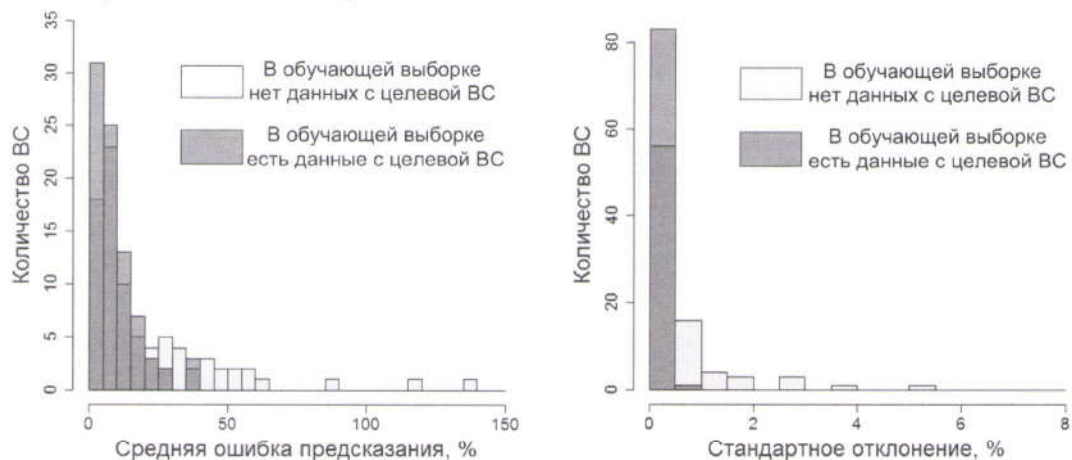


Рисунок 5. Гистограммы распределения предсказания времени умножения матриц

На рис. 5 представлены гистограммы количества ВС в зависимости от средней ошибки предсказания и стандартного отклонения предсказания. Для подавляющей части вычислительных систем ошибка прогноза не превосходит 30%, а разброс ошибок получаемых предсказаний не превосходит 2%.

В диссертации представлены сравнительные результаты оценки качества выбора наиболее эффективной реализации матричного умножения из библиотек Intel MKL и OpenBLAS в зависимости от используемой стратегии. Для оценки качества выбора вводится показатель L , который определяет процент увеличения среднего времени умножения матриц относительно оптимального выбора

$$L = \frac{100}{n} \sum_{i=1}^n \frac{\sum_{j=1}^{e_i} (\overline{y}_{i,j} - \min(y_{i,j}^{MKL}, y_{i,j}^{OpenBLAS}))}{\sum_{j=1}^{e_i} \min(y_{i,j}^{MKL}, y_{i,j}^{OpenBLAS})},$$

где $\overline{y}_{i,j}$ - время работы выбранной реализации.

Простые стратегии выбора приводят к существенному увеличению среднего времени умножения матриц относительно оптимального выбора:

- если на всех ВС использовать OpenBLAS, то увеличение составит 32%,
- если на всех ВС использовать Intel MKL – 13%,
- если на системах Intel использовать Intel MKL, а на AMD OpenBLAS – 11%.

Использование предсказания времени выполнения для выбора наиболее эффективной реализации приводит к увеличению среднего времени умножения матриц относительно оптимального выбора всего на 2%. Эксперименты с остальными алгоритмами показали, что увеличение среднего времени решения задачи происходит не более чем на 6%.

В **третьей главе** представлено описание программного комплекса сборки и исполнения программ, разработанных по технологии быстрого прототипирования.

Комплекс предназначен для сборки программ, написанных с использованием технологии быстрого прототипирования, и корректного их выполнения с использованием наиболее эффективных реализаций алгоритмов. Алгоритмическую основу для выбора наиболее эффективной реализации составляет предложенный метод, построенный на основе методов машинного обучения. Основным режимом функционирования среды исполнения – вычисления на рабочих станциях.

При проектировании архитектуры программного комплекса сборки и исполнения программ решались следующие задачи:

- Трансляция макрокоманд.
- Интеграция в существующие средства разработки программ, такие как Microsoft Visual Studio.

- Автоматическое определение доступных реализаций алгоритмов, соответствующих макроописанию, и конвертеров типов данных для последующего использования.
- Автоматическое преобразование форматов хранения в случае необходимости.
- Сбор накапливаемой информации о времени работы реализаций алгоритмов и конвертеров типов данных для последующего выбора наиболее оптимальной по заданному макроописанию.
- Автоматический выбор наиболее оптимальной реализации алгоритма по заданному макроописанию с учётом временных затрат на преобразование форматов хранения типов данных, использующихся в алгоритме. При необходимости может осуществляться ручной выбор.
- Корректная обработка ошибочных ситуаций, которые могут произойти при выполнении макрорасширений, и возврат к выполнению пользовательского кода.

Как результат, была разработана архитектура системы, состоящая из трёх уровней: уровень сборки, уровень планирования и уровень исполнения. Уровень исполнения представляет собой набор динамических библиотек выполняющих преобразование форматов хранения с помощью конвертеров и сопряжение интерфейсов с помощью адаптеров. На уровне планирования, используя доступные конвертеры и адаптеры уровня исполнения, планировщик выполняет определение наиболее эффективной реализации и корректную обработку ошибочных ситуаций. На уровне сборки осуществляется трансляция исходных кодов программ, написанных с использованием макроязыка и подготовка их к дальнейшей сборке стандартными средствами. В работе приводится детальное описание каждого из уровней архитектуры системы.

Программный комплекс сборки и исполнения программ, разработанных по технологии быстрого прототипирования, реализован в среде разработки Microsoft Visual Studio 2012 с использованием технологии объектно-ориентированного программирования. Реализация классов и структур данных компонент уровней исполнения и планирования выполнена на языке программирования высокого уровня C++. Иерархия классов содержит 38 классов. Алгоритмы машинного обучения, использующиеся планировщиком, написаны на языке программирования для статистической обработки данных R. Реализация макропроцессора выполнена на языке Python для обеспечения максимальной совместимости между программными платформами. Расширение для Microsoft Visual Studio, выполняющее интеграцию макропроцессора в среду разработки, реализовано на языке C#. Общий объем программного комплекса составляет около 5000 строк кода.

В четвертой главе описана методика применения технологии быстрого прототипирования, представлены примеры использования, демонстрирующие эффективность технологии и описаны прикладные задачи, решенные с помощью предложенного подхода.

В разделе 4.1 представлена методика применения технологии быстрого прототипирования, специфичная для разных уровней пользователей: прикладной программист, системный администратор, инженер по сопровождению.

Инженер по сопровождению библиотеки выполняет реализацию адаптеров в соответствии с требованиями к их реализации, освобождая основных разработчиков библиотеки от данной работы. В том случае, если в библиотеке используются специфичные структуры данных, то для них потребуется реализация конвертеров. Разработанные адаптеры и конвертеры поставляются вместе с библиотекой. На практике эту роль может выполнять разработчик библиотеки. Системный администратор выполняет установку библиотек и настройку среды исполнения на конкретной вычислительной системе. Программист выполняет макроописание прикладной программы на макроязыке, выполняет сборку и запуск полученной программы. При необходимости он может выбрать используемую библиотеку и реализацию, которую необходимо использовать, иначе она будет определена автоматически во время работы программы.

В разделе 4.2 представлены примеры использования технологии быстрого прототипирования на задачах умножения матриц и вычисления быстрого преобразования Фурье.

В разделе 4.3 приведены результаты исследования трудозатрат при разработке и модификации программного обеспечения.

Для оценки эффективности использования технологии быстрого прототипирования выполнено исследование трудозатрат при разработке и модификации ПО на примере задачи умножения матриц. Для исследования было привлечено 94 студента факультета вычислительной математики и кибернетики Нижегородского государственного университета им. Н.И. Лобачевского. Использование технологии быстрого прототипирования позволило сократить время разработки программ почти в 2 раза, а время миграции на библиотеку может быть сокращено более чем в 4 раза. Заметим, что время реализации алгоритма умножения матриц сильно зависит от навыков участника, в то время как при использовании технологии быстрого прототипирования уровень подготовки участников незначительно сказывается на времени разработки.

В разделе 4.4 приведены результаты оценки эффективности работы планировщика и накладные расходы, которые он вносит в работу программы.

Накладные расходы на использование технологии быстрого прототипирования на тестовой вычислительной системе (процессор Intel Core i7-3820 3.6 ГГц, 16 ГБ оперативной памяти) не превосходят 1 мс. Это позволяет без существенной потери эффективности решать задачи средней и большой размерности. Эффективного выполнения задач малой размерности можно достигнуть за счёт программной оптимизации средств исполнения программ и устранения зависимости от внешних сервисов.

В разделе 4.5 приведены результаты апробации подхода при решении задачи разработки унифицированного интерфейса для переупорядочивателей при решении СЛАУ. Результатом выполненной работы является программная сис-

тема, которая обеспечивает использование требуемого переупорядочивателя (METIS, O-MATRUZ, MORSY) для решателя СЛAУ без пересборки проекта. Использование данной системы позволило осуществлять выбор наиболее подходящего переупорядочивателя для различных модельных задач в ручном или автоматическом режиме. Работа поддержана грантом (соглашение от 27 августа 2013г. № 02.В.49.21.0003 между МОН РФ и ННГУ).

В разделе 4.6 представлены результаты использования предложенного подхода в НИИ измерительных систем им. Ю.Е. Седакова. Приведены результаты апробации программного комплекса при создании программной системы «Кристалл», предназначенной для решения задач планирования процесса изготовления больших интегральных схем с субмикронными топологическими нормами. Использование технологии быстрого прототипирования привело к сокращению трудозатрат на разработку программной системы.

В **заключении** сформулированы основные результаты диссертационной работы.

ОСНОВНЫЕ РЕЗУЛЬТАТЫ РАБОТЫ

1. Разработан новый метод построения оценки времени выполнения программ, основанный на методах машинного обучения.
2. Предложена новая технология быстрой разработки прототипов при необходимости выбора из множества близких по функциональности компонент.
3. Разработан и реализован программный комплекс, обеспечивающий сборку и исполнение программ, реализованных с использованием предлагаемого подхода быстрой разработки прототипов.
4. Подтверждена практическая полезность предложенного подхода результатами исследования трудозатрат при разработке и модификации программного обеспечения.
5. Подтверждена практическая полезность разработанного комплекса апробацией при создании программной системы «Кристалл», предназначенной для решения задач планирования процесса изготовления больших интегральных схем с субмикронными топологическими нормами, в НИИ измерительных систем им. Ю.Е. Седакова.

ПУБЛИКАЦИИ ПО ТЕМЕ ДИССЕРТАЦИОННОЙ РАБОТЫ

Публикации в научных журналах, рекомендованных ВАК.

1. Сиднев А.А., Гергель В.П. Автоматический выбор наиболее эффективных реализаций алгоритмов // Вычислительные методы и программирование: новые вычислительные технологии. – 2014. – Т. 15, № 4. – С. 579-592. – 0,88/0,8 п.л.
2. Сиднев А.А., Гергель В.П. Методы и программные средства макромодульной разработки программ // Вестник нижегородского университета им. Н.И. Лобачевского. – 2012. – №5(2). – С. 294-300. – 0,4/0,2 п.л.

Остальные публикации по теме диссертационной работы.

3. Сиднев А.А. О подходах к решению задачи миграции программного обеспечения // Прикладная математика и фундаментальная информатика. – 2015. – № 2. – С. 288-293. – 0,375 п.л.

4. Sidnev A. Runtime prediction on new architectures // In Proceedings of the 10th Central and Eastern European Software Engineering Conference in Russia (CEE-SECR '14). – New York, USA: ACM, 2014. – Article 17, 7 pp. – 0,44 п.л.

5. Сиднев А.А., Гергель В.П. Применение макромодульного подхода к разработке программ // Вестник ПНИПУ. Электротехника, информационные технологии, системы управления. – 2014. – № 12. – С. 52-66. – 0,937/0,8 п.л.

6. Сиднев А.А. Методы и программные средства макромодульной разработки программ // Научный сервис в сети Интернет: многообразие суперкомпьютерных миров: Труды Международной суперкомпьютерной конференции. – М.: Изд-во МГУ, 2014. – С. 157-159. – 0,1 п.л.

7. Сиднев А.А., Гергель В.П. Применение макромодульного подхода разработки программ // Высокопроизводительные параллельные вычисления на кластерных системах. Материалы XIV международной конференции. – Пермь: Изд-во Пермского национального исследовательского политехнического университета, 2014. – С. 393-400. – 0,5/0,4 п.л.

8. Sidnev A.A., Gergel V.P. Macromodule technology // Proceedings of the 4th Spring/Summer Young Researchers' Colloquium on Software Engineering (SYRCoSE 2010). – Nizhny Novgorod, Russia, 2010. – P. 26-28. – 0,2/0,1 п.л.

9. Сиднев А.А., Гергель В.П. О применении макромодульной технологии разработки программ // Высокопроизводительные параллельные вычисления на кластерных системах. Материалы X международной конференции. Том 1. – Пермь: Изд-во Пермского государственного технического университета, 2010. – С. 172-177. – 0,37/0,18 п.л.

10. Сиднев А., Горшков А. Влияние загрузки шины данных на ускорение параллельных программ // Технологии Microsoft в теории и практике программирования. Материалы конференции / Под ред. проф. В.П. Гергеля. – Нижний Новгород: Изд-во Нижегородского госуниверситета, 2010. – С. 93-97. – 0,3/0,15 п.л.

11. Сиднев А.А., Козинев Е.А. Построение моделей теоретического времени выполнения алгоритма, учитывающих количество кэш-промахов и латентность для последовательных программ // Технологии Microsoft в теории и практике программирования. Материалы конференции / Под ред. проф. В.П. Гергеля. – Нижний Новгород: Изд-во Нижегородского госуниверситета, 2009. – С. 205-210. – 0,37/0,18 п.л.

Свидетельства о государственной регистрации программ для ЭВМ.

12. Сиднев А.А. Свидетельство о государственной регистрации программ для ЭВМ № 2015610070 «Среда исполнения макромодульных программ ММТ» от 12 января 2015 г.